





F7LY内核介绍

2025年全国大学生计算机系统能力大赛 操作系统设计赛(全国) OS内核实现赛道

小队成员：曹子宸、郑喆宇、官恺祺

指导老师：蔡朝晖



一、概述

1.1 整体架构

1.2 参考的项目

二、内核详细介绍

2.1 机器启动

2.2 中断处理

2.3 内存管理

三、亮点与总结

2.4 进程管理

2.5 文件系统

2.6 进程间通信

3.1 工作亮点

3.2 经验总结

3.3 总结与展望

目

录

1893

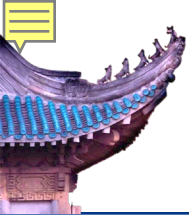
武汉大学



01 整体架构

Overall Architecture

自强、弘毅、求是、拓新



1.1 整体概述



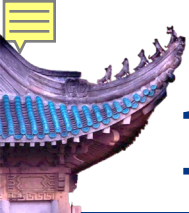
#	用户名	队伍	rank
1	T202518123995755	rustflyer/ 哈尔滨工业大学 (深圳)	18173.0000
2	T202510213995926	火箭队/ 哈尔滨工业大学	16672.9033
3	T202518123995568	Chronix/ 哈尔滨工业大学 (深圳)	13935.2477
4	T202510003995291	undefined/ 清华大学	12334.2201
5	T202510486995158	武大前锋/ 武汉大学	12080.3687
6	T202510486995205	F7LY/ 武汉大学	11595.8546

在github上有完整的开发记录，截止目前共946次commit记录，独立编码，kernel共九万八千行代码。

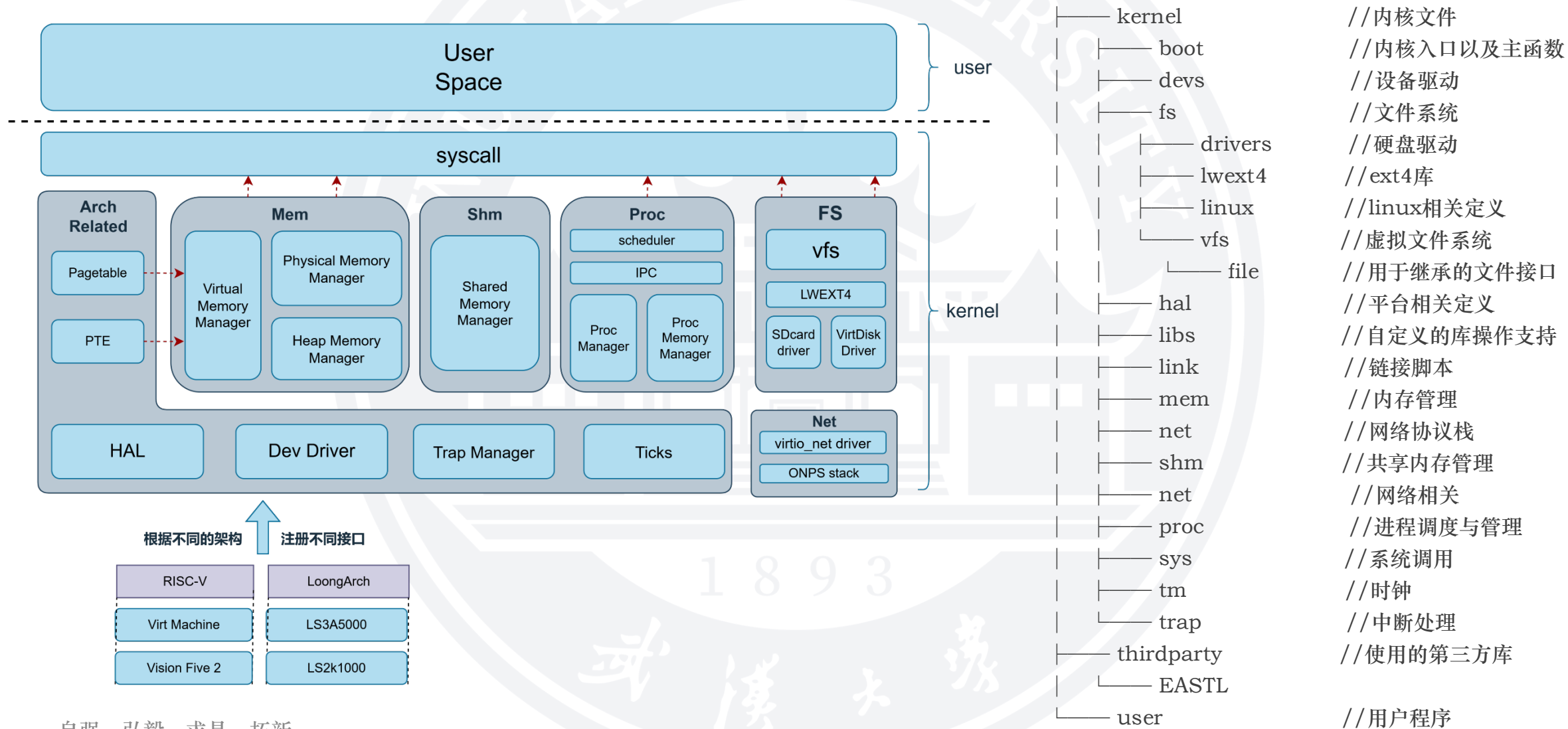
高分通过决赛阶段初赛测例,截止8.17晚23:59,排行榜排名第六

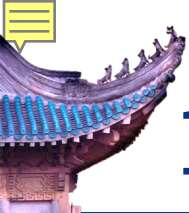
自强、弘毅、求是、拓新





1.1 整体架构





1.2 参考的代码



◆ 2024武汉大学队伍XN6

- LA架构的硬件抽象层以及页表管理方法

◆ LWEXT4

- 移植c语言ext4库

◆ MIT-xv6

- 中断管理以及内核布局

◆ Open-NPStack

- 移植开源网络协议栈

◆ 内存分配器

- 采用LibAllocator和Buddy算法

◆ 2023华中科技大学队伍AVX

- 移植了sdcard驱动

◆ EASTL

- 作为第三方库

◆ Linux

- 参考系统调用规范实现

02 内核详细介绍

Complete Introduction for F7LY kernel



2.1 机器启动

◆ RISC V启动流程

- 基于SBI(Supervisor Binary Interface)规范，Bootloader工作在M mode，内核代码从entry.S开始运行在S mode
- 启动流程遵循经典的多阶段启动模式： Bootloader
entry.S start.cc main.cc

- F7LY OS支持双架构启动，机器启动的源文件分别在kernel/boot/riscv/ 和 kernel/boot/loongarch / 目录下

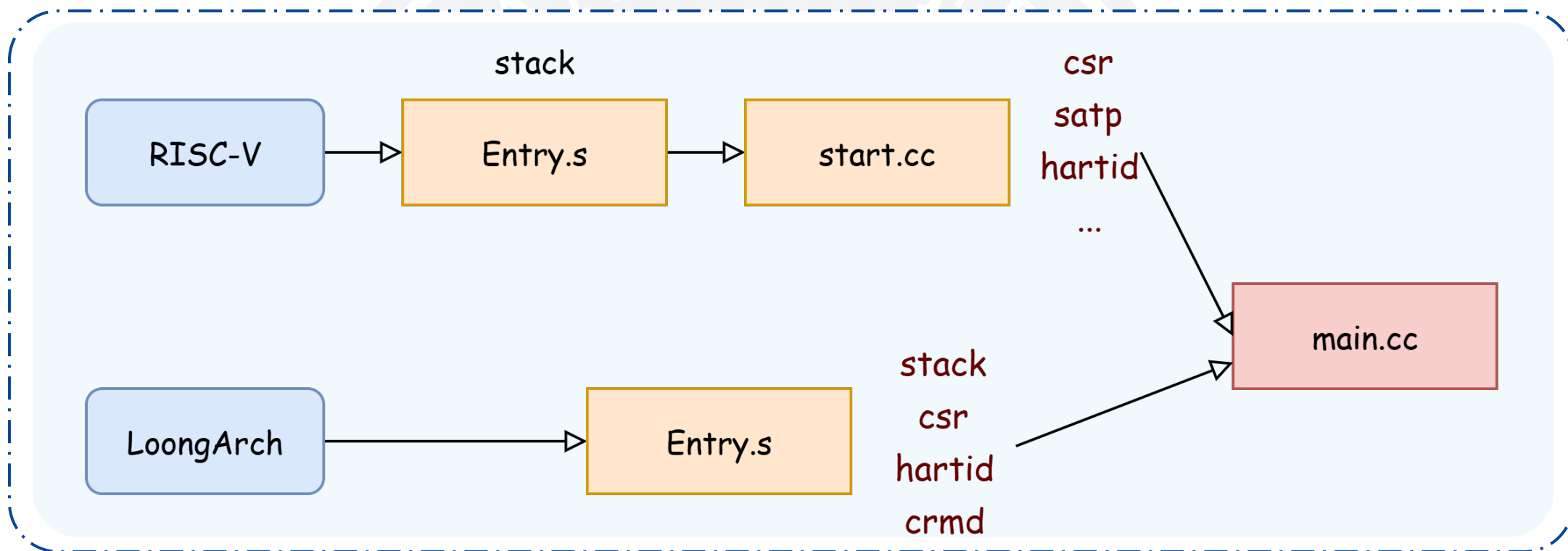
◆ LoongArch启动流程

- 采用LoongArch原生启动方式，通过ACPI等标准接口进行硬件初始化
- loongarch部分将start.cc阶段合并到entry.S中，在其中完成设置设备操作空间、设置指令数据访问空间，CSR寄存器的初始化，以及栈空间的设定，并开启fpu浮点数指令。

- 整个kernel可执行文件的入口点在各自架构的entry.S中的_entry标签，通过链接脚本指定



2.1 总体逻辑路线



Rv架构三级启动，主要硬件初始化位于`start.cc`，`entry.S`起一个跳板的作用。
LA架构硬件初始化较多，在`Entry.s`中全部完成而后统一进入`main`函数进入内核初始化

- 统一的抽象层
- 架构特定优化
- 模块化设计
- 面向对象实现



2.2 中断管理器

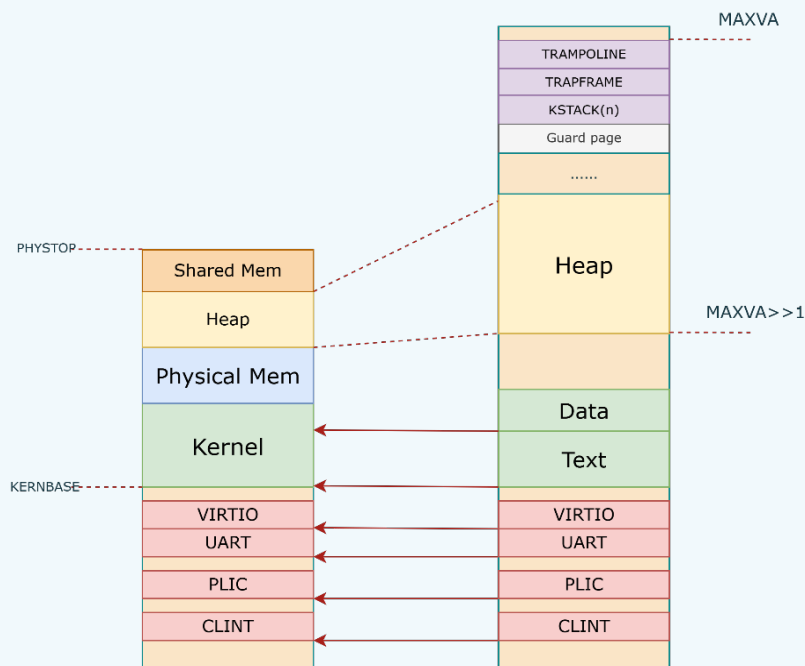
```
1
2 class trap_manager
3 {
4 public:
5     void init();
6     void inithart();
7     int devintr(); // 处理外部中断和软件中断
8     void usertrap(); // 用户态中断处理
9     void usertrapret(); // 用户态返回处理
10    void machine_trap();
11    void kerneltrap(); // 内核态中断处理
12 private:
13    void timertick(); // 时钟中断处理
14    SpinLock tickslock; // 保护ticks的自旋锁
15    uint ticks; // 时钟中断计数
16    uint timeslice; // 时间片计数
```

F7LY在其底层根据RISC V和loongarch的不同硬件设备使用而封装了不同的中断处理器，在不同文件夹下实现了PLIC、EXTIOI、APIC的驱动并应用在中断处理之中。

- RISC V在处理中设定了中断处理的入口（uservec）以及返回的入口(usertrapret)，具体逻辑参照了xv6的中断处理，但封装在对象之中，并使用包装函数（wrap_前缀的全局函数）设置中断入口。
- Loongarch针对架构的特殊性设置了TLB重填的处理程序入口（tlbrefill.S），以及机器异常的处理程序（merror.S），使用类似的逻辑进行接口统一化，使得上层程序可以统一调用。

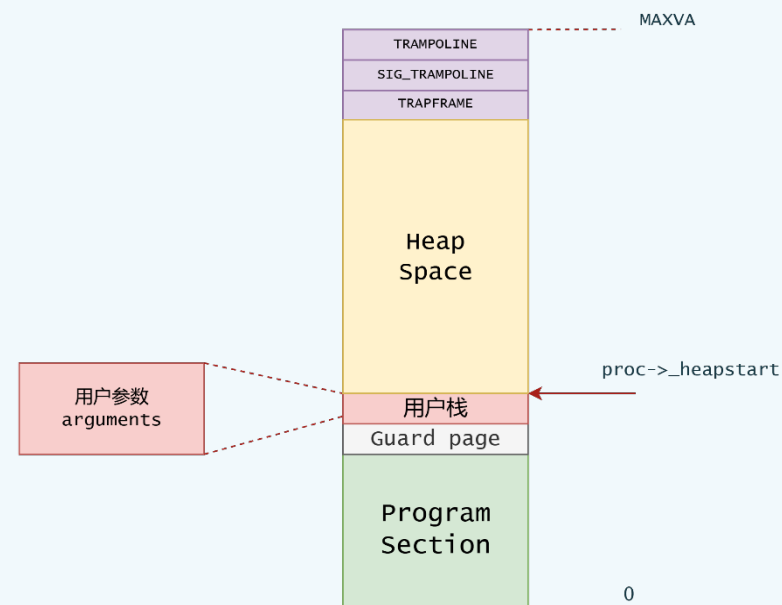
2.3 内存管理——地址空间

内核地址空间



从KERNBASE开始映射内核映像；最顶层的64MB设定为共享内存。堆内存从顶开始向下增长，剩余的空闲物理地址交由物理内存管理器管理由下往上增长。

用户地址空间



对于用户地址空间，在加载时会区分程序段、用户栈、用户堆、Trampoline等进行加载。从基础加载地址开始

2.3 内存管理——物理内存管理器

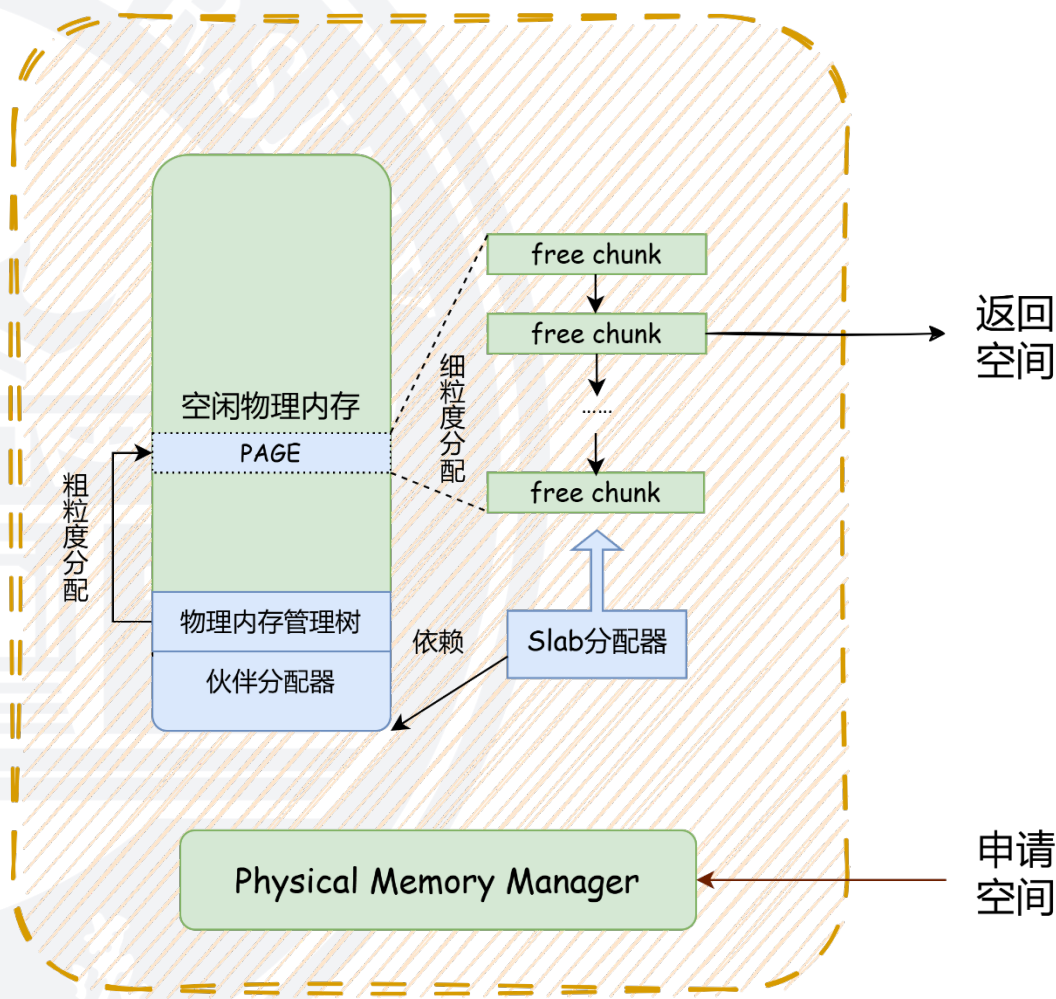
物理内存管理类PhysicalMemoryManager

使用粗细粒度混合分配管理内核所需的物理内存，
统一使用伙伴内存分配器进行粗粒度分配。

更细粒度的大小分配（函数 kmalloc ）采用了
linux的 SlabAllocator

目的就是缓存一些被释放的object因此这些基础的
structures在多次调用期间被预留起来。

这部分物理内存用于各类用户请求的空间



物理内存管理器结构图



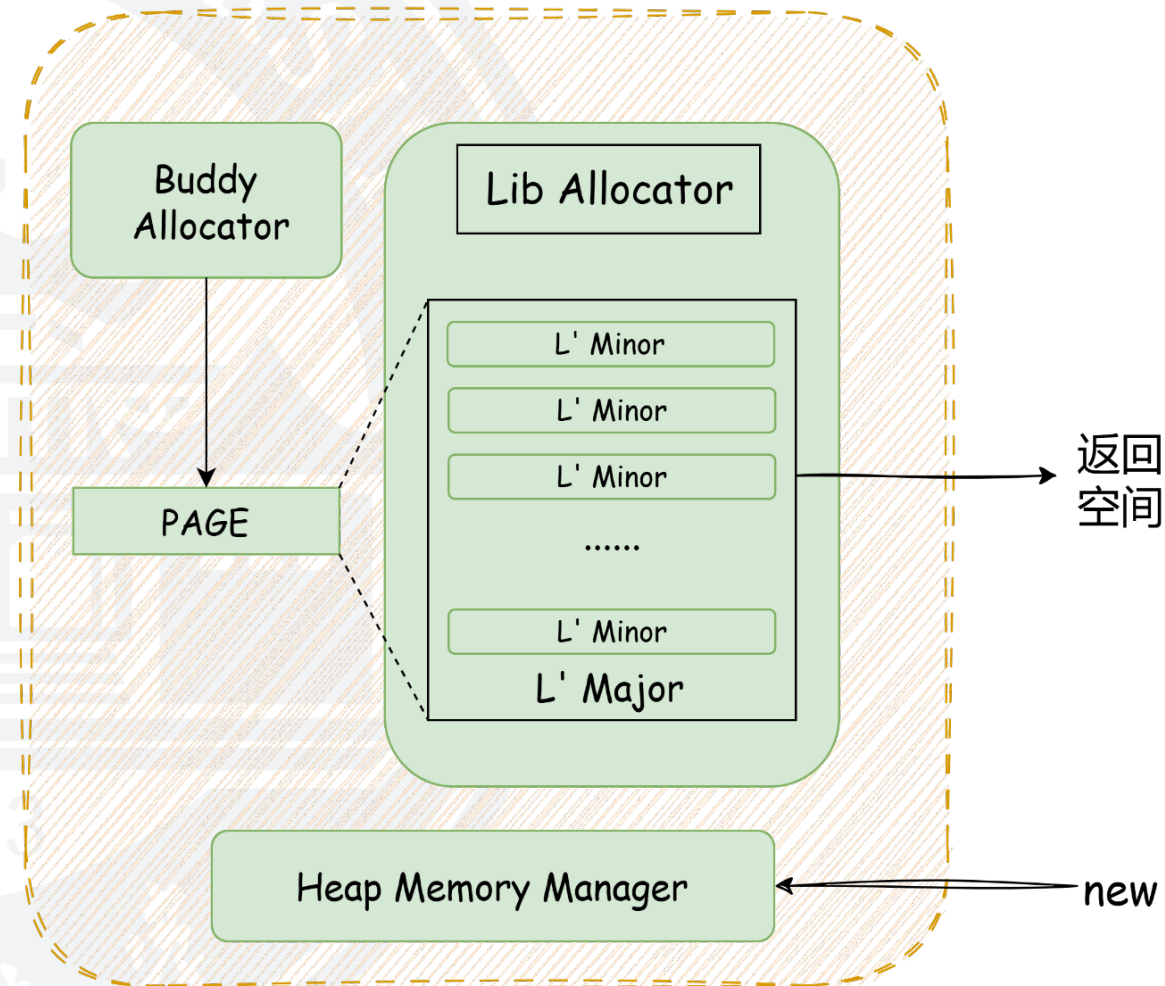
2.3 内存管理——堆内存管理器

堆内存管理类HeapMemoryManager

使用粗细粒度混合分配管理内核所需的物理内存，使用伙伴内存分配器进行粗粒度分配。

堆内存主要用于内核的构建，支持cpp的动态内存分配的new和delete运算符

```
void * operator new(uint64 size)
{
    void *p = mem::k_hmm.allocate(size);
    return p;
}
```





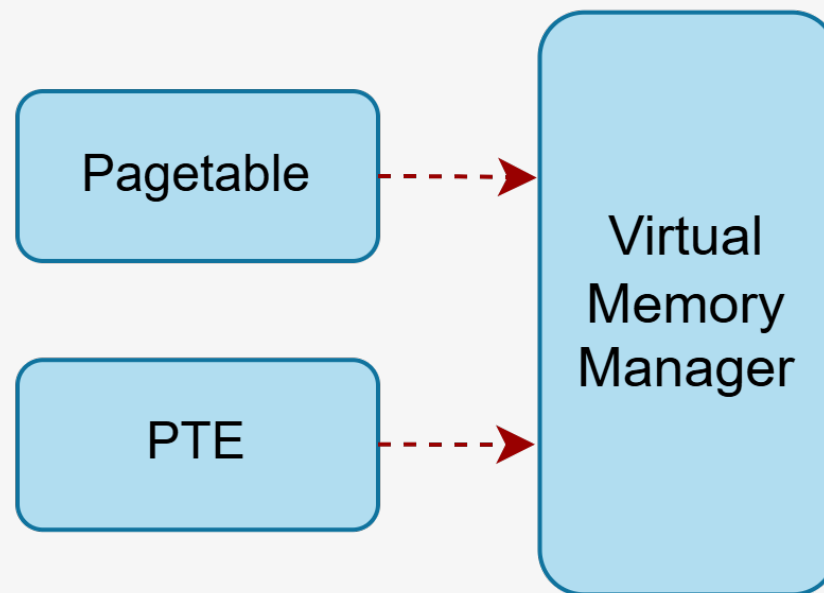
2.3 内存管理——虚拟内存管理器

F7LY采用静态预配置+管理器分派的机制进行地址空间管理。

- 核心由 VirtualMemoryManager 管理器统一负责虚拟地址的分配、映射与回收。
- 底层则由 PageTable 类协助完成页表操作与维护。

主要接口：

- `map_page 映射虚拟地址
- `allocate_vma_page 为vma惰性分配页面
- `copy_in/copy_out 内核态和用户态传递



虚拟内存的管理结构

页表与PTE分架构，设置不同函数



2.4 进程管理——进程控制块

F7LY此处统一使用Pcb结构体来记录任务，可以通过标志位决定任务的具体属性，而不需要区分进程结构体和线程结构体。

```
class Pcb;  
extern Pcb k_proc_pool[num_process];
```

- ProcessMemoryManager

进程内部的内存管理器

- 进程的标志

如gid\uid\pid\ppid等

- 进程的打开文件

Ofile以及文件的标志

- 进程的属性

时间统计、资源限制、上下文信息

- 信号处理

信号函数、屏蔽掩码、待处理掩码

- 进程状态

可运行、睡眠、kill等

关键字
段

2.4 进程管理——进程调度

调度器类代码

```
class Scheduler
{
private:
    SpinLock _sche_lock;
public:
    Scheduler() = default;
    void init( const char *name );    //调度器初始化
    void switch_to_proc(Pcb* p);    //调度切换函数
    int get_highest_proirity();    //获取当前队列中最高优先级进程的进程号
    void start_schedule();    //最开始的调度

    void yield();    //进程放弃CPU，状态从RUNNING改为RUNNABLE
    void call_sched();    //正常调度开始
};
```

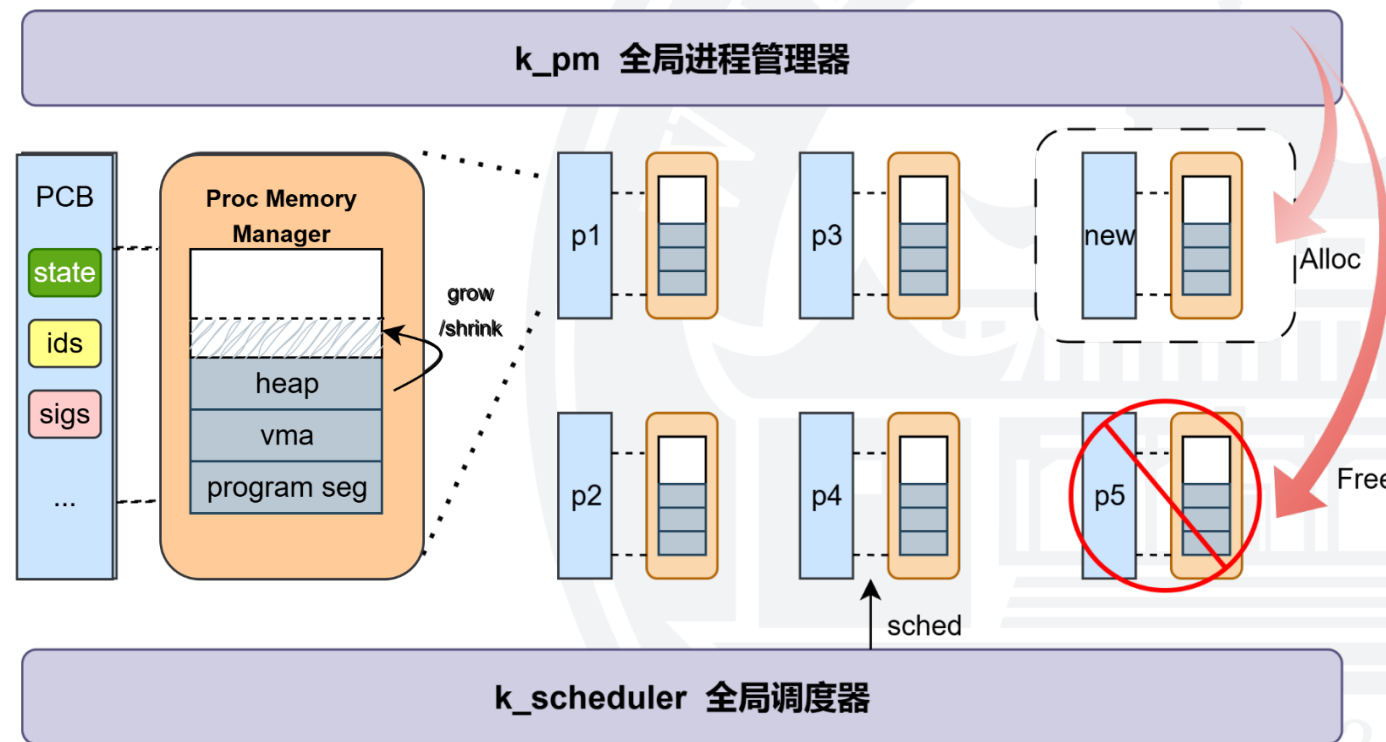
使用全局对象 k_scheduler
其中具体的调度方法使用了与MIT xv6类似的逻辑方法。

Scheduler类承担了原本c语言书写的xv6中的调度任务。

循环调度

这种用对象封装两个有栈协程 call_sched 和 start_schedule 的方法可以使F7LY比xv6有更清晰和高效的调度方法。

2.4 进程管理——双管理器

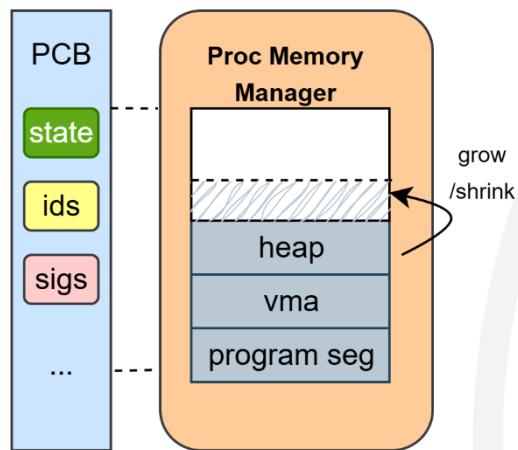


F7LY的进程管理使用
双管理器模式。

类ProcessManager封装全局
对进程的许多操作，对所有进
程全局管理

类ProcessMemoryManager
用于管理单个进程的内存，封
装整个进程地址空间的增长、
共享等

2.4 进程管理——内存管理器

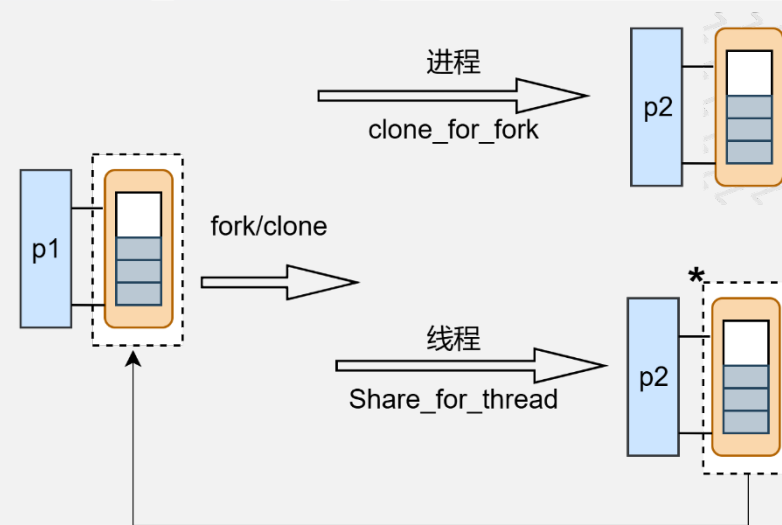


```
class ProcessMemoryManager
{
public:
    // 程序段管理
    program_section_desc
    → prog_sections[max_program_section_num];
    int prog_section_count;
    // 堆内存管理
    uint64 heap_start;
    uint64 heap_end;
    // 页表管理
    mem::PageTable pagetable;
    // VMA 管理
    VMA vma_data;
    // 共享标志
```

对程序段、堆空间的管理
Vma的分配释放
Execve时清空内存
处理共享的引用计数

复制进程

内存管理器会根据fork时传入的标志不同处理。
线程共享内存会传入原来管理器的指针。
进程不共享内存会深度拷贝一个管理器。

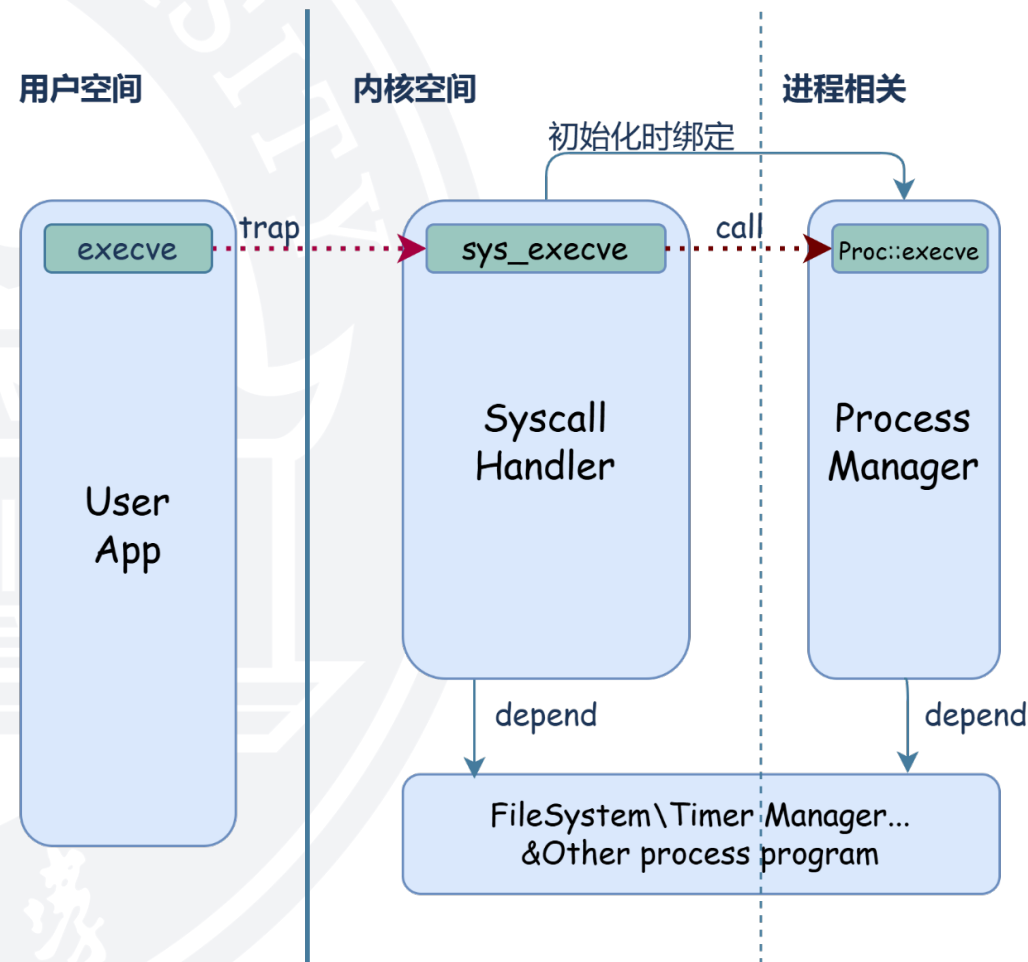


2.4 进程管理——系统调用

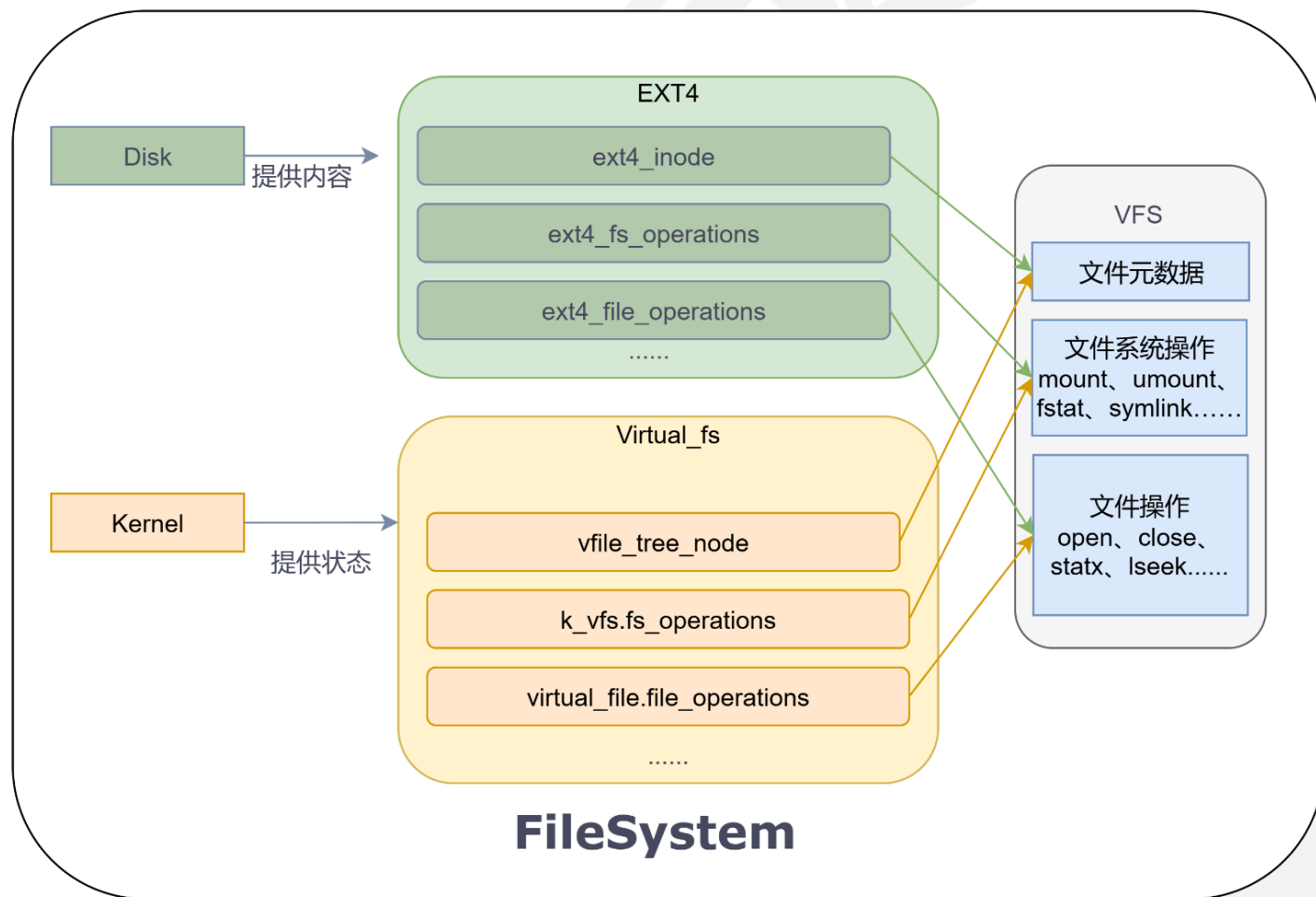
```
class SyscallHandler
{
    // 使用了这个类的成员函数指针
    using SyscallFunc = uint64 (SyscallHandler::*); // 定义一个函数指针类型 SyscallFunc
private:
    SyscallFunc _syscall_funcs[max_syscall_funcs_num]; // 存储系统调用函数的指针数组
    const char *_syscall_name[max_syscall_funcs_num]; // 存储系统调用名称的指针数组
    uint64_t _default_syscall_impl(); // 默认的系统调用实现
    .....
}

extern SyscallHandler k_syscall_handler;
```

在系统调用流程中，SyscallHandler 的入口首先接收用户态传递的参数，然后分发到对应的系统调用处理函数。如果系统调用涉及进程管理相关操作（如 `fork`、`execve`、`wait` 等），SyscallHandler 会调用 `ProcessManager` 负责实际的逻辑执行



2.5 文件系统——虚拟文件系统



F7LY系统移植了lwext4库并封装了一层vfs。

由于文件类型不同，我们同时支持了一层虚拟文件管理器，用于访问系统文件

通过这一层封装来选择正确的文件类型与操作。



2.5 文件系统——虚拟文件架构

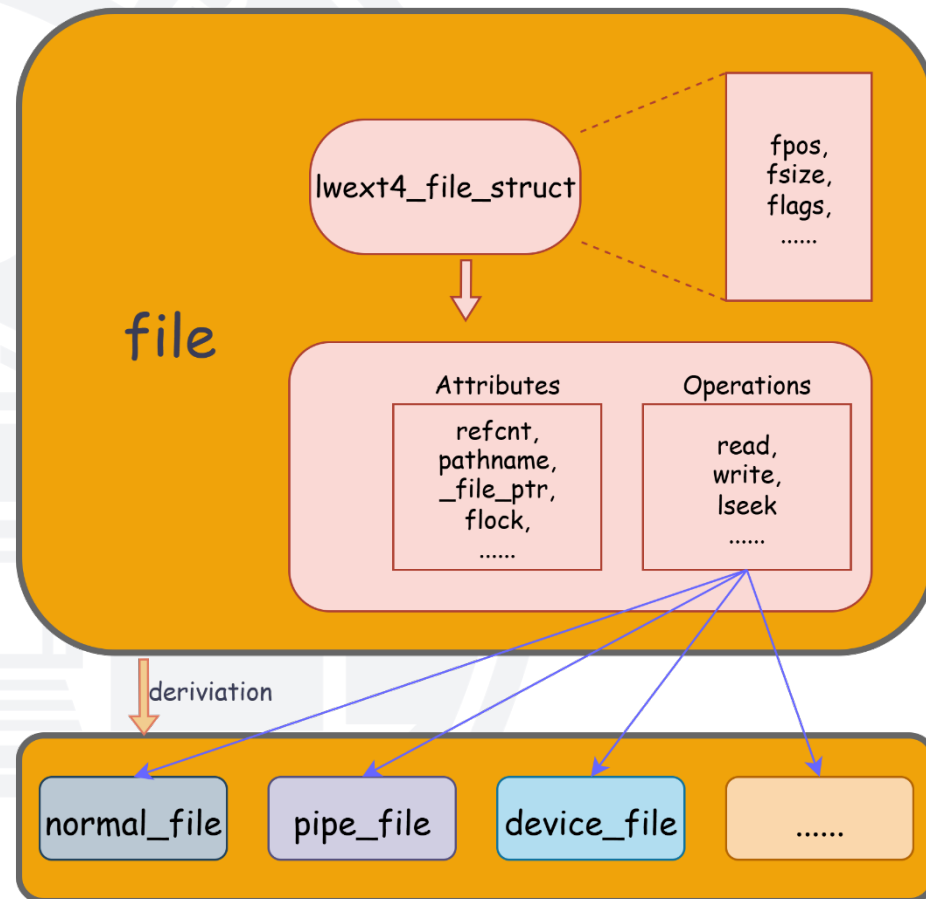
在操作系统中一切皆文件。

依据文件类型的不同派生出不同的类型并实现对应的操作

底层元数据使用了lwext4结构体

虚拟文件类file表示进程中打开文件的状态属性

派生结构用于不同类型的文件访问与操作

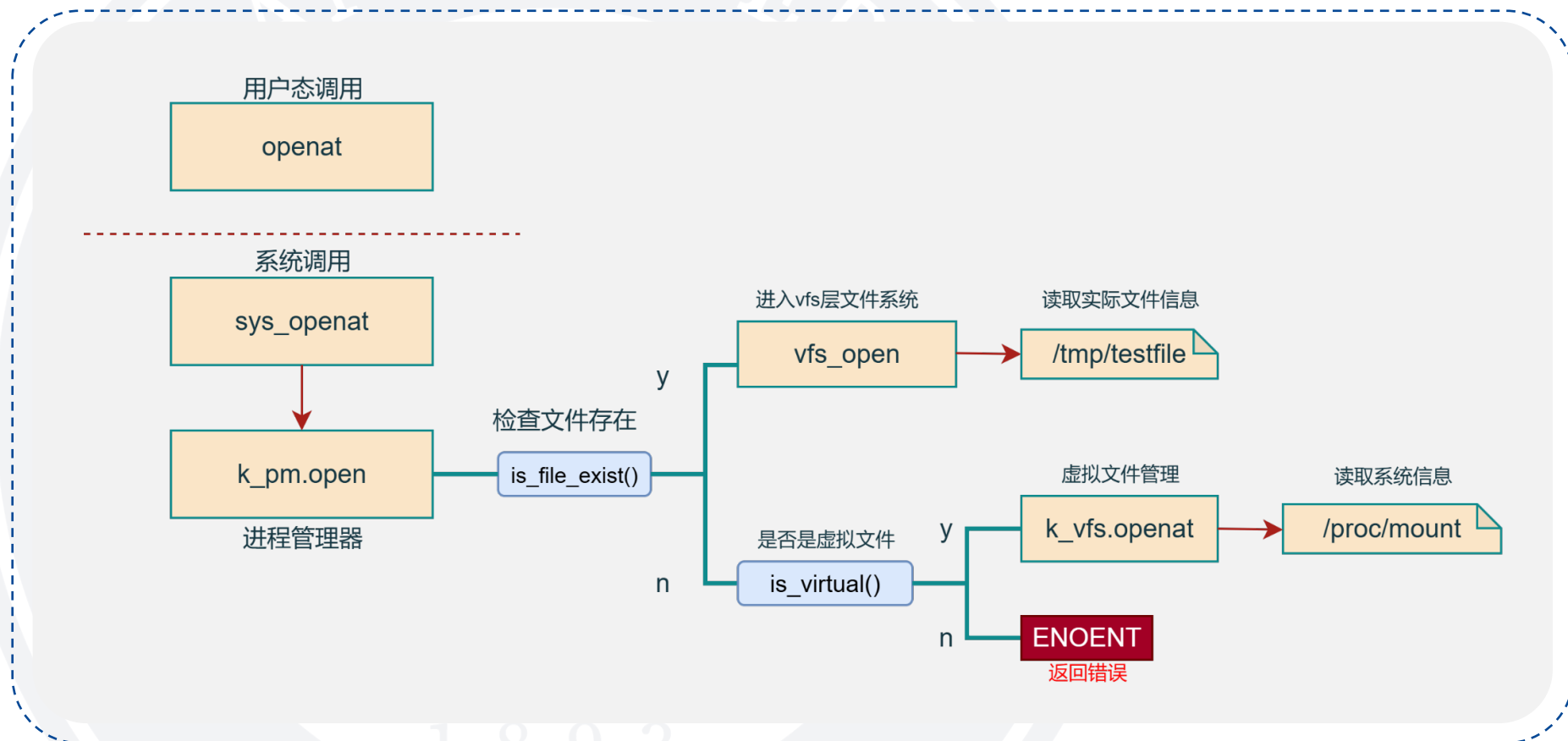


虚拟文件的继承结构



2.5 支持的文件操作

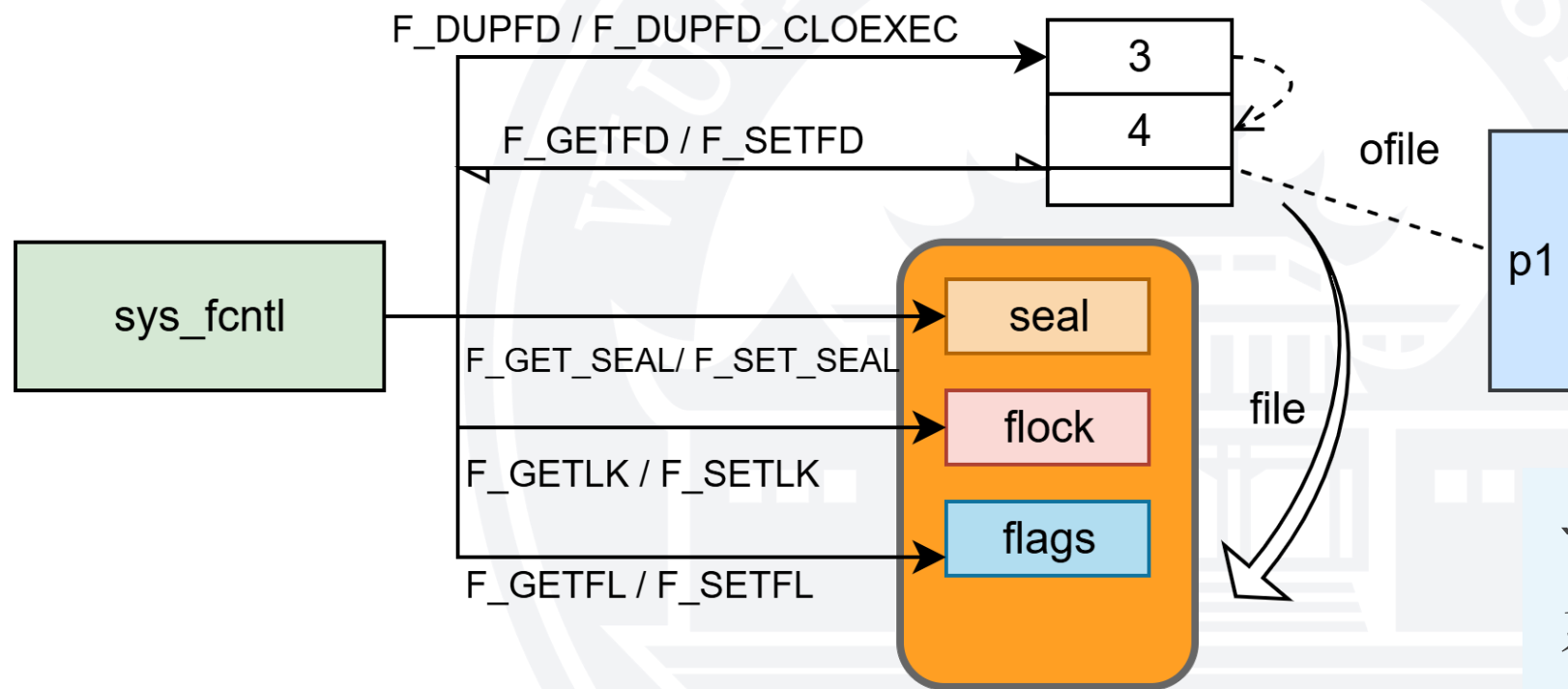
read/write
open
fstat
getdent
mkdir
rename
ftruncate
fallocate
chmod
chown
symlink
link
lseek
.....



两层封装的作用在于选择正确的文件操作



2.5 支持的文件操作



‘fcntl’ 系统调用在 UNIX/Linux 系统中用于对文件描述符进行多样化的管理。F7LY 目前已经支持若干常用的 ‘op’ 操作。



2.6 进程间通信——信号机制

F7LY 借鉴 Linux 的设计，支持符合 POSIX 标准的信号机制，并在 PCB 中内置信号处理所需的关键数据结构。

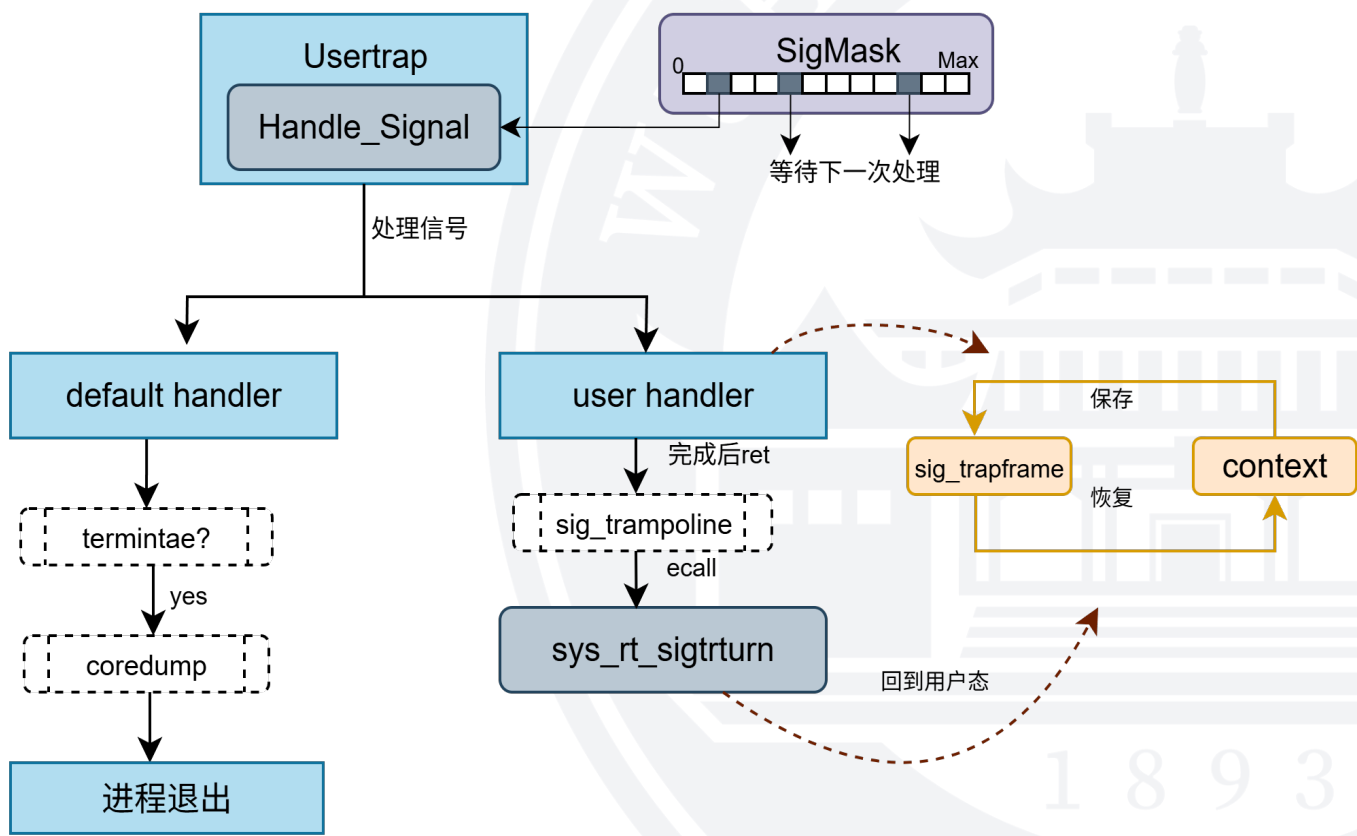
- `signal_frame` 用于保存信号处理过程中的进程上下文；
- `sigaction` 定义每个信号对应的处理行为。

```
namespace signal
{
    struct signal_frame
    {
        sigset_t mask;
        TrapFrame tf;
        signal_frame *next;
    };
    typedef struct sigaction
    {
        void (*sa_handler)(int); // 信号处理函数
        sigset_t sa_mask;        // 处理期间阻塞的信号
        int sa_flags;
    } sigaction;
}
```

信号相关结构体代码



2.6 进程间通信——信号机制



在进行信号处理时，F7LY会根据各个信号对应的`sigaction`，动态判断进行默认信号处理或用户自定义信号处理

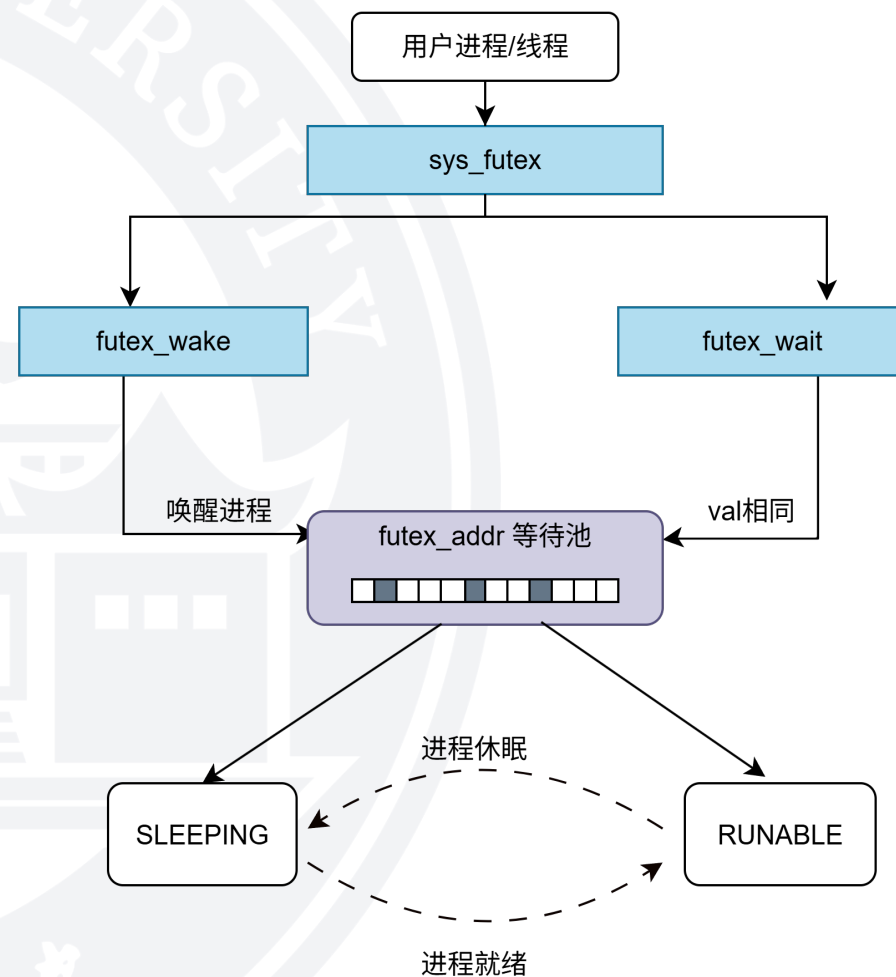
对于默认信号处理，F7LY会根据POSIX标准的信号语义，判断是否进行`terminate`和`coredump`。

对于用户自定义信号，F7LY实现了完整的处理函数调用和三参数传递，并使用哨兵保证栈空间安全。

2.6 进程间通信——Futex

Futex是Linux内核提供了一种高效用户态同步原语，可用于实现互斥锁、条件变量、信号量等多种同步机制。

F7LY实现的futex系统调用遵循Linux标准，提供了`futex\_wait`和`futex\_wakeup`两个核心操作，实现了高效的用戶态同步机制。



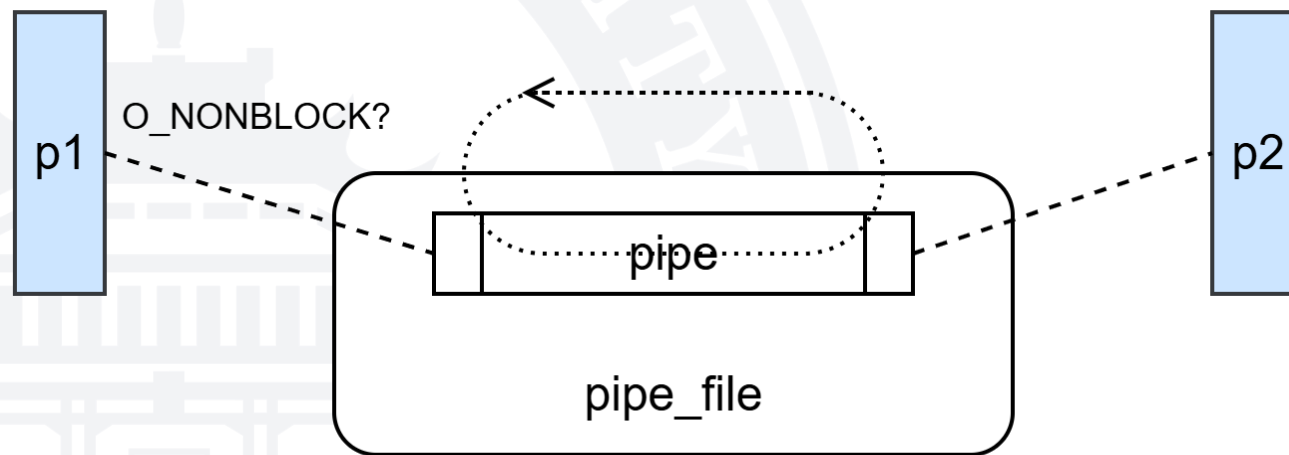
futex 工作机制示意图

2.6 进程间通信——管道

管道在进程中为一个打开文件，文件类型为pipe_file

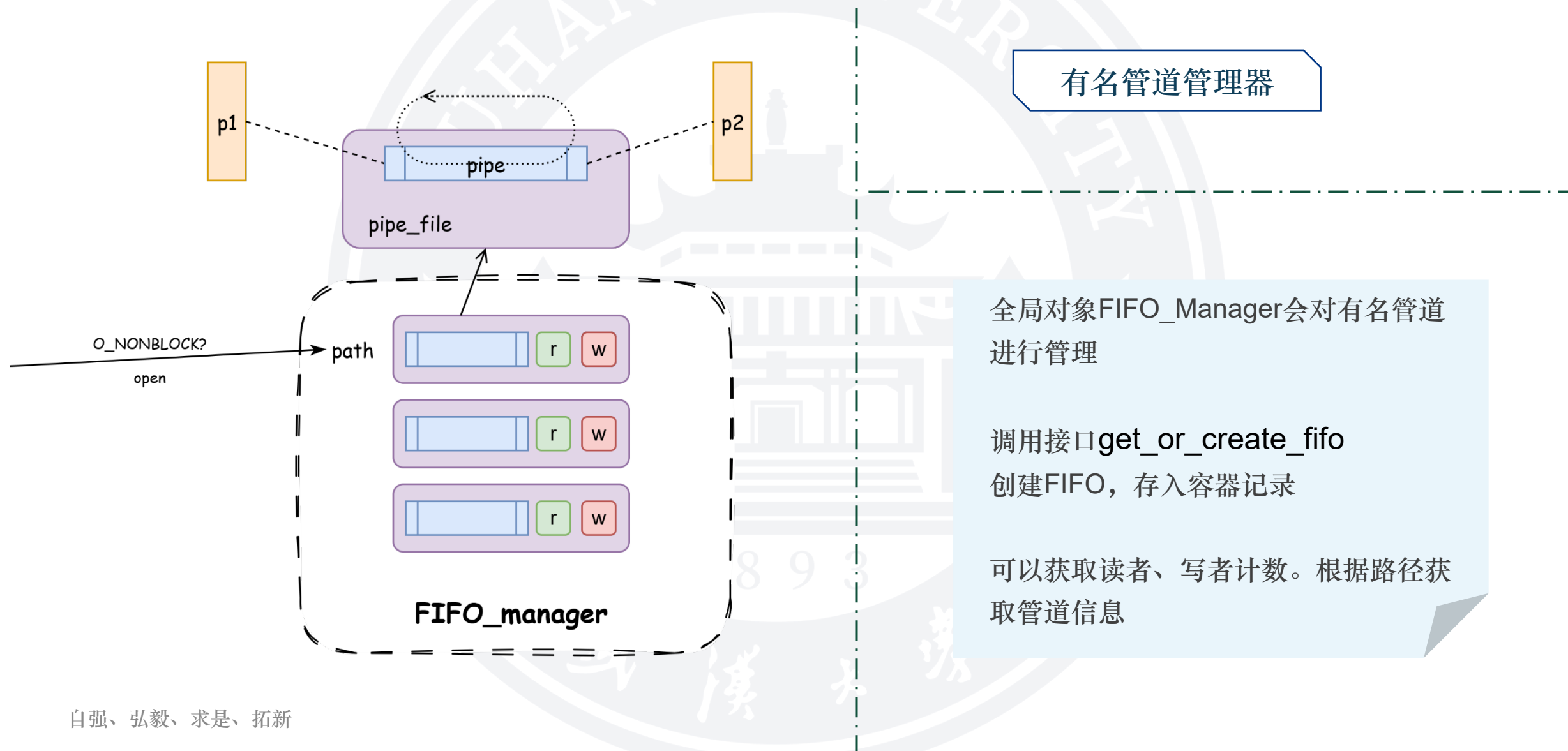
pipe_file拥有一个pipe作为类成员

管道通过循环缓冲的方法传输文件，可通过打开标志决定是否阻塞

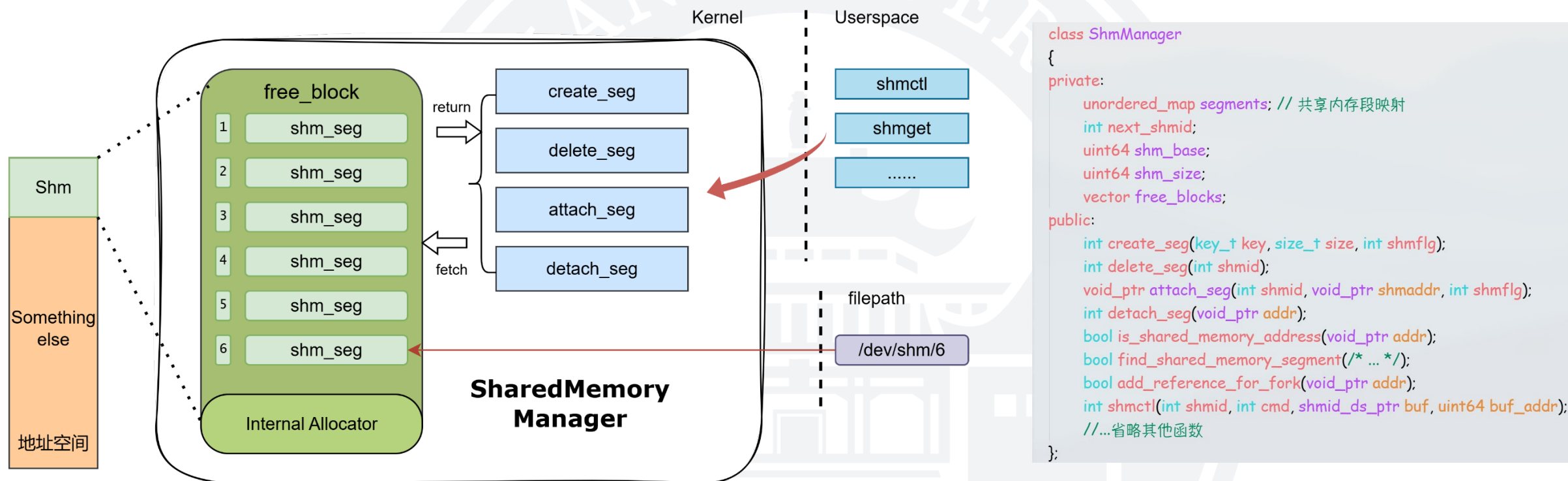


拥有有名管道的pipe_file会记录路径名。用户可通过路径名访问，也是FIFO的标志之一

2.6 进程间通信——管道



2.6 进程间通信——共享内存



共享内存由全局管理器统一分配。
可适配shm系列系统调用
可以支持mmap系统调用共享内存段的映射
也通过/dev/shm访问共享内存

其中管理的最小单位linux兼容的shm_segment

每个段存储其映射的虚拟地址和相应进程号

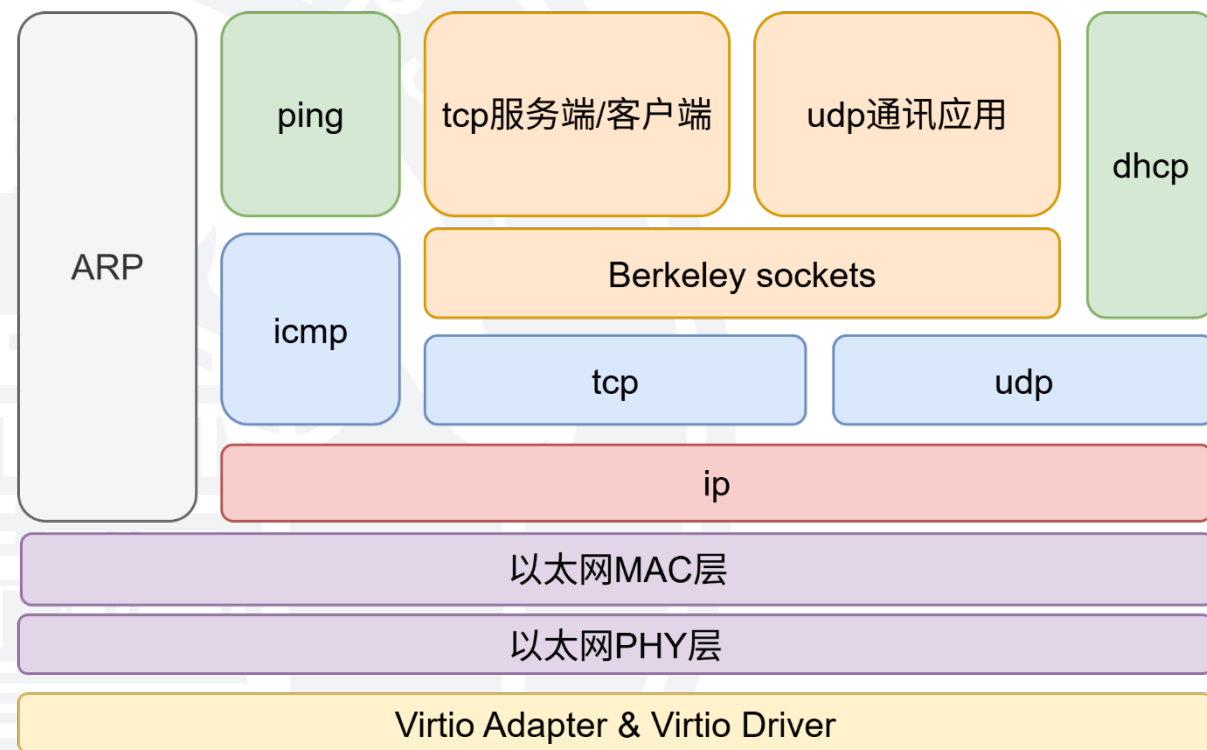


2.7 网络模块——分层架构

网络系统基于 Open-NPStack 协议栈实现，旨在构建高效灵活的网络通信能力。本系统支持 AF_INET、AF_UNIX 等多种地址族的套接字操作，完整实现了 TCP 与 UDP 传输协议。网络系统通过统一的抽象接口管理所有网络设备与套接字资源。

整个网络系统主要包含以下几个核心组件：

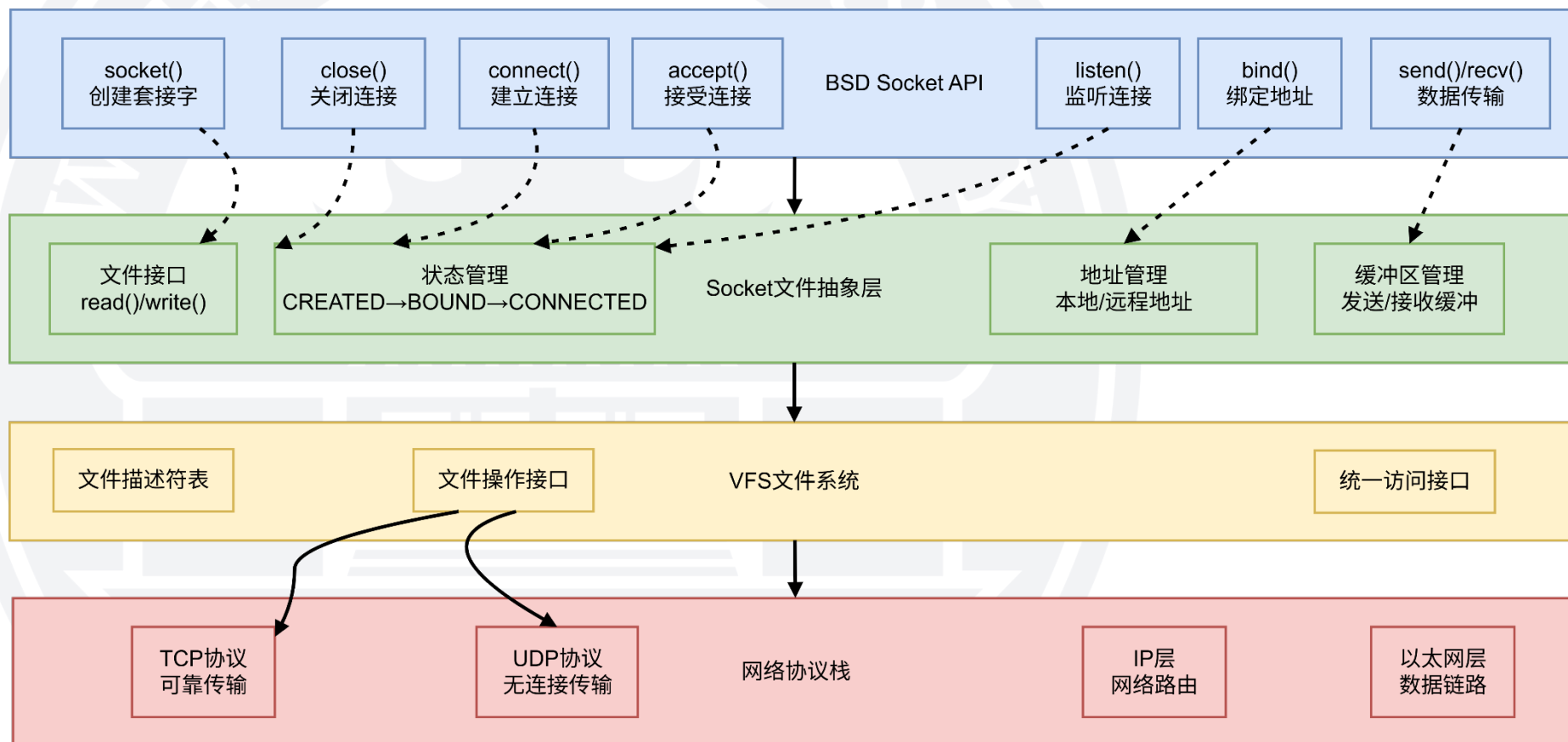
1. VFS 集成层
2. BSD Socket 接口
3. 网络协议协议栈
4. 网络适配层
5. VirtIO 网络驱动层



网络系统采用分层架构设计，从底层硬件驱动到上层应用接口，形成了完整的网络协议栈。

2.7 网络模块——标准BSD Socket接口

F7LY-OS实现了完整的POSIX标准BSD Socket接口，为用户程序提供标准化的网络编程API。通过VFS文件系统抽象，Socket被视为特殊文件，支持统一的文件操作接口，实现了“一切皆文件”的设计理念。

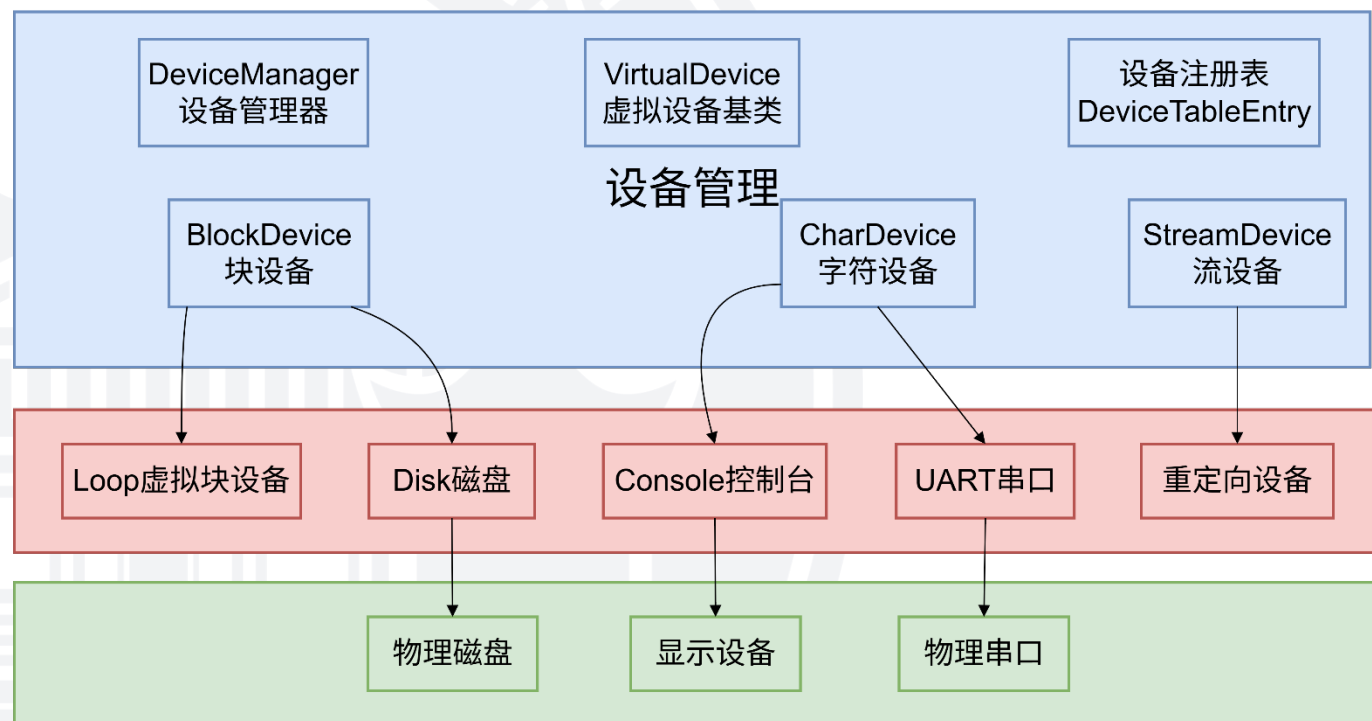




2.8 设备管理

F7LY-OS的设备管理体系以模块化为核心，采用分层与面向对象设计，统一了多种硬件设备的管理与访问接口。通过定义虚拟设备基类、字符设备、块设备、流设备等，系统实现了对不同类型设备的支持和调度。

设备管理器集中负责设备的注册、查找、删除等生命周期管理，保障了系统资源的合理分配和设备操作的安全性。具体设备如磁盘、网络、UART和Loop设备等，均通过统一架构实现了高效协作和扩展性。



03 亮点与总结

Highlight & Summary of F7LY kernel



3.1 工作总结（初赛阶段）

双架构驱动

- ① RISC-V ,
LoongArch 的 硬
件抽象驱动
- ② 完成合逻辑的
中断处理程序。

面向对象设计

- ①完成进程管理和内存管理模块
- ②支持线程任务的创建与调度
- ③实现底层架构无关性松耦合结构

虚拟文件系统

- ①支持EXT4与FAT32
- ②支持sdcard驱动

适配Busybox 和 libc

- ①基础实现进程间通信（IPC）
机制和信号处理功能
- ②补充和完善系统调用接口与相
关功能
- ③构建多等级调试日志，方便问
题定位

3.1 工作重点（决赛阶段）

进程间通信

- ①构建更完善的进程间通信
- ②支持socket\pipe\shm\signal\memfd等机制

重构内存布局

- ①添加共享内存区域
- ②使用独立内存管理器规划进程地址空间

添加线程支持

完善文件系统

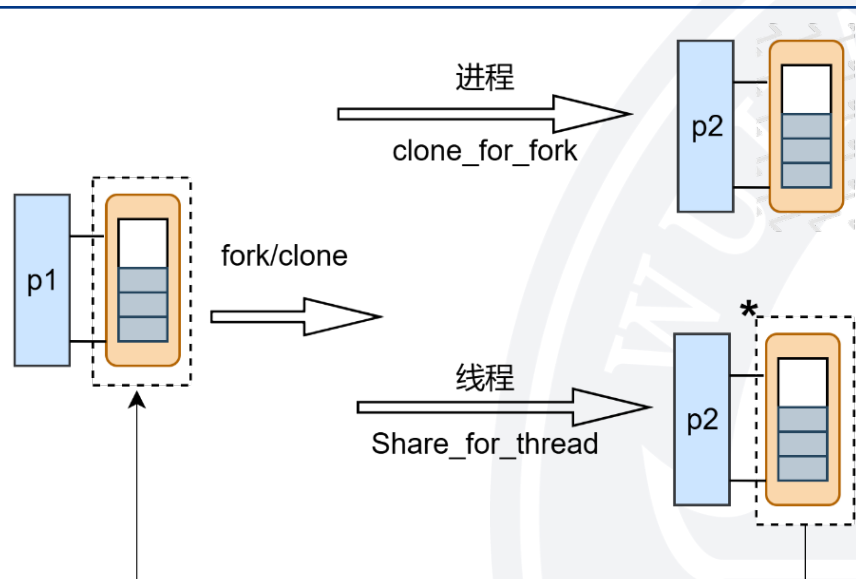
- ①移植lwext4库
- ②封装vfs层转发操作调用

规范系统调用

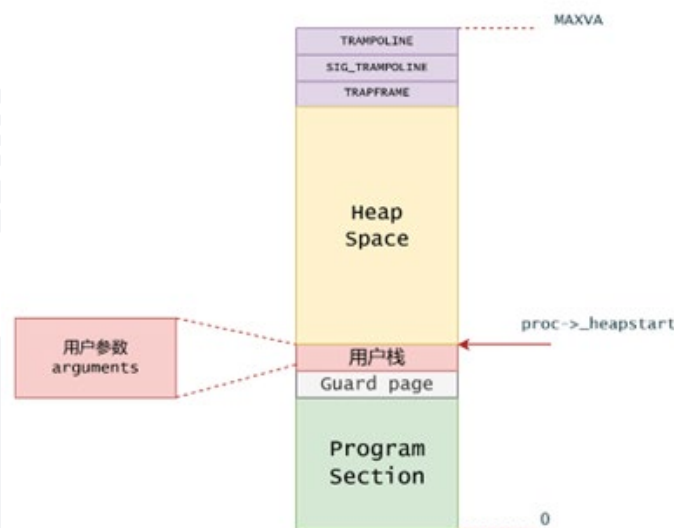
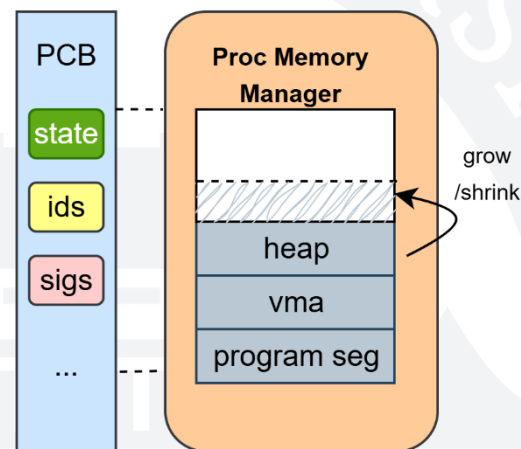
- ①根据LTP的要求完善错误码
- ②按照linux manual的规范实现了220余个系统调用



3.2 完成亮点——线程支持/内存规划

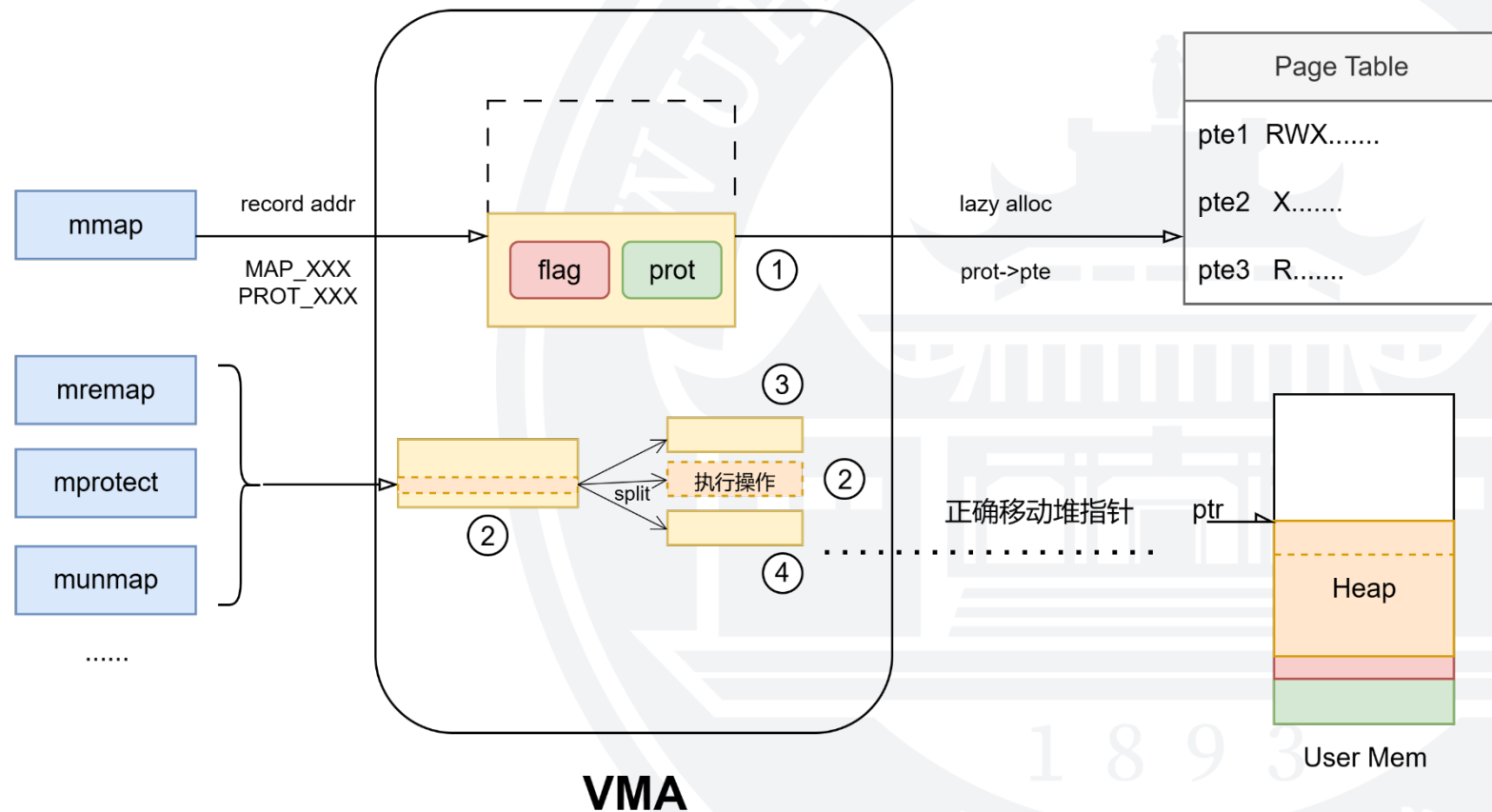


- 共享内存分配释放由进程内存空间管理器自动管理
- 线程由clone flag组合创建



- 进程地址空间用程序段处理而不是sz的粗放式管理
- 单独管理器自动处理引用
- 拥有:
 - 用户标识uid/gid
 - 进程组pgid/sid
- 相关处理将在权限检查、信号分发、资源隔离等地方

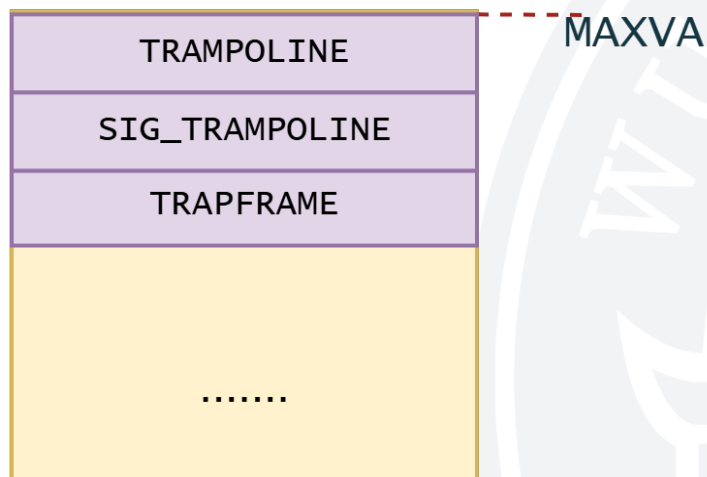
3.2 完成亮点——内存规划



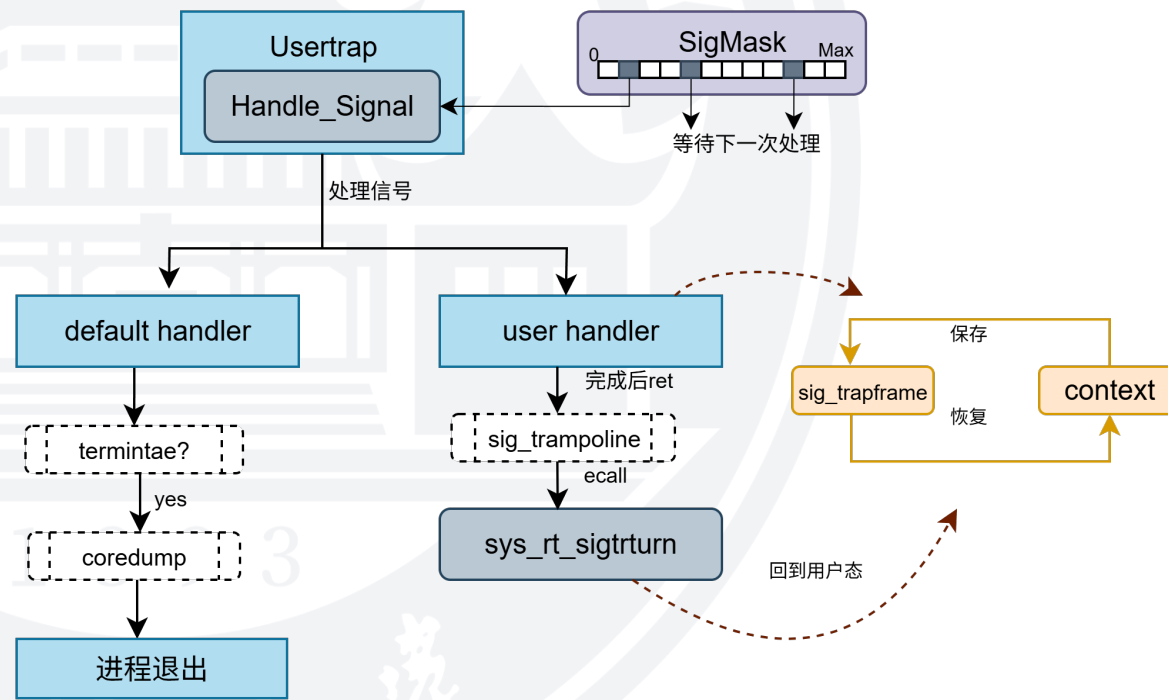
- 内存权限管理 :合理赋予权限, 使用platform统一不同架构标志位
- Vma: 懒分配
- 权限分配: 根据传入prot合理赋予权限
- Vma操作: 拆分原有vma, 扩展或收缩堆指针。
- 引用机制完善

3.2 完成亮点——信号处理

支持自定义信号处理函数，用户空间三参数传递

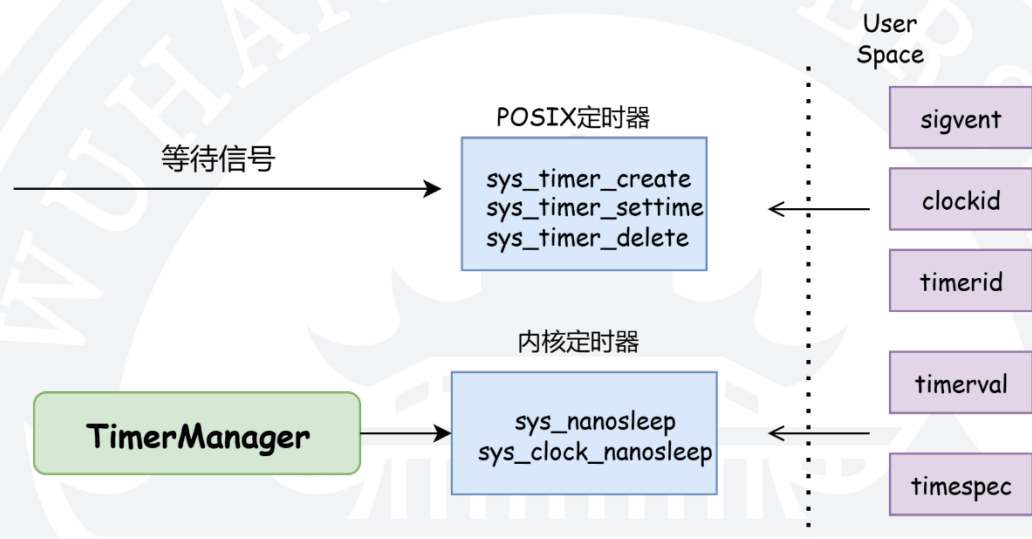


拥有信号跳板栈，
保证栈空间安全



3.2 完成亮点——时钟处理

- 拥有内核自己的定时器
- 同时兼容posix定时器的功能
- 可以根据用户传入的结构进行解析，实现定时功能



同时支持POSIX标准的各种时钟类型，具有精细的时钟管理
可满足不同的语义和使用场景。

```
enum SystemClockId : uint
{
    /// @brief 系统实时时钟，可被系统管理员调整
    /// 受系统时间设置影响，可能出现时间跳跃
    CLOCK_REALTIME = 0,

    /// @brief 单调时钟，从系统启动开始单调递增
    /// 不受系统时间调整影响，适用于测量时间间隔
    CLOCK_MONOTONIC = 1,

    /// @brief 进程CPU时间，测量调用进程消耗的CPU时间
    /// 包括用户态和内核态的执行时间
    CLOCK_PROCESS_CPUTIME_ID = 2,

    /// @brief 线程CPU时间，测量调用线程消耗的CPU时间
    /// 仅计算当前线程的执行时间
    CLOCK_THREAD_CPUTIME_ID = 3,

    /// @brief 原始单调时钟，不受NTP调整影响
    /// 提供更"原始"的单调时间，不经过频率调整
    CLOCK_MONOTONIC_RAW = 4,

    .....

    /// @brief 启动时间闹钟，基于BOOTTIME的闹钟
    /// 可以在系统挂起时基于启动时间触发
    CLOCK_BOOTTIME_ALARM = 9,

    /// @brief SGI周期计数器（已废弃）
    /// 原驱动已移除，此ID保留作为占位符，不应重用
    CLOCK_SGI_CYCLE = 10,

    /// @brief 国际原子时钟（TAI）
    /// 基于原子时标准，不包含闰秒调整
    CLOCK_TAI = 11,

    /// @brief 最大时钟数量限制
    MAX_CLOCKS = 16
};
```

3.3 经验总结——概述

1. 积累从零开始构建内核的系统化经验，包括：

- 完成机器引导；
- 初始化串口输出；
- 搭建内存布局；
- 编写中断处理；
- 实现最小可用内核；
- 逐步完善核心功能。

2. 系统调用的实现结合《System Calls Manual》和参考其他优秀项目成果，符合 Linux 原生 raw syscall 规范。

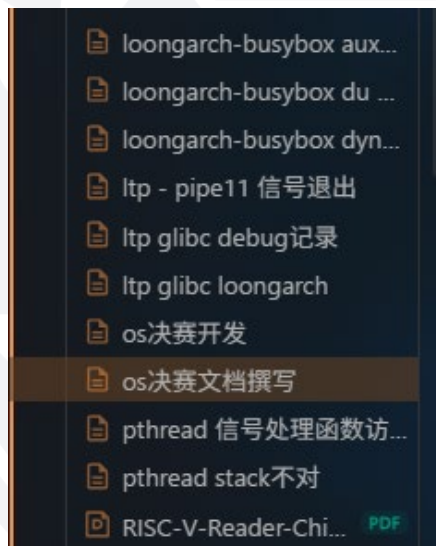
3. 不断积累调试经验，包括：

- 用户态程序的单步调试；
- 利用 GDB 跟踪调用栈；
- 精准定位出错变量；
- 依靠完善的日志获取运行关键信息。

3.3 经验总结——问题记录



仔细记录每一次调试过程，形成记录文档，遇到重复问题时可查阅



```
/**
 * @brief 释放堆内存
 *
 * 释放进程堆内存占用的所有页面，将堆大小重置为0。
 *
 * 注意：此函数内部调用cleanup_heap_to_size(0)，仅在free_all_memory()中内部调用
 */
void free_heap_memory();

/**
 * @brief 清理堆内存到指定大小
 * @param new_size 新的堆大小，如果为0则完全释放堆内存
 * @return true 成功，false 失败
 */
bool cleanup_heap_to_size(uint64 new_size);

/*****
 * VMA管理接口
 *
 * 说明：VMA管理现在统一通过free_all_memory()进行，不建议单独调用
 *****/
```

详细为函数书写注释。使用doxygen风格注释，方便使用时参考和后续查阅。未测试\未完成的代码明确标注TODO，便于后续发现问题。

```
#elif defined(LOONGARCH)
/// TODO:未测试正确性
void *mem;
uint64 a;
// printfCyan("[vmalloc] oldsz
if (newsz < oldsz)
```

3.3 经验总结——调试日志

```
// 使用匿名命名空间限制 disable_printf 变量的作用域
namespace {
    bool disable_printf_flag = false; // 默认关闭 printf
}

void Printer::enable_printf()
{
    disable_printf_flag = false;
}

void Printer::disable_printf()
{
    disable_printf_flag = true;
}

void Printer::print( const char *fmt, ... )
{
    // 如果禁用了 printf 输出, 则直接返回
    if (disable_printf_flag) {
        // 仍需要处理可变参数以避免潜在问题
        va_list ap1;
        va_start(ap1, fmt);
        va_end(ap1);
        return;
    }
}
```

可在特定测例中使用打印使能, 只观察单个测例的调试输出

```
#define printfRed(format, ...) k_printer.print("
#define printfGreen(format, ...) k_printer.print(
#define printfBlue(format, ...) k_printer.print("
#define printfCyan(format, ...) k_printer.print('
#define printfYellow(format, ...) k_printer.print
#define printfWhite(format, ...) k_printer.print
#define printfMagenta(format, ...) k_printer.pr
// 颜色太少了, 我给你加几个
#define printfBlack(format, ...) k_printer.print(
#define printfOrange(format, ...) k_printer.prin
#define printfPurple(format, ...) k_printer.print
#define printfPink(format, ...) k_printer.print("
#define printfBrown(format, ...) k_printer.print
#define printfGray(format, ...) k_printer.print('
#define printfLightRed(format, ...) k_printer.pr
#define printfLightGreen(format, ...) k_printer.
#define printfLightBlue(format, ...) k_printer.p
#define printfLightCyan(format, ...) k_printer.p
#define printfLightYellow(format, ...) k_printer
#define printfLightMagenta(format, ...) k_print
// Background colors
#define printfBgRed(format, ...) k_printer.print
#define printfBgGreen(format, ...) k_printer.pr
#define printfBgBlue(format, ...) k_printer.print
#define printfBgYellow(format, ...) k_printer.pr
#define printfBgCyan(format, ...) k_printer.prin
#define printfBgMagenta(format, ...) k_printer.
```

打印输出有多种色彩
便于观察。警告使用
黄色、完成使用绿色、
错误信息使用红
色.....

使用多层printf开关,
可灵活控制是否打开
调试输出。Panic的输
出不会被隐藏, 运行
时问题会及时暴露

3.3 经验总结——用户态调试经验

调试中一般使用三种手段：
查看测例源码、
对比qemu-riscv64-static和qemu-loongarch64-static的标准系统调用、gdb调试

```
static struct tcase {
    int *fdin;
    loff_t *offin;
    int *fdout;
    loff_t *offout;
    int exp_errno;
} tcases[] = {
    { &badfd, NULL, &pipes[1], NULL, EBADF },
    { &pipes[0], NULL, &badfd, NULL, EBADF },
    { &wrfd, NULL, &pipes[1], NULL, EBADF },
    { &pipes[0], NULL, &appendfd, NULL, EINVAL },
    { &rdfd, NULL, &wrfd, NULL, EINVAL },
    { &pipes[0], &offset, &wrfd, NULL, ESPIPE },
    { &rdfd, NULL, &pipes[1], &offset, ESPIPE },
};
```

部分测例可以直接察看到预期行为，可以根据测试的期望进行修改

更多的测例不能直接看到问题，我们可以使用wsl运行标准的内核，使用-strace对比每一个系统调用的行为和返回值

```
26466 wait4(26468,0x555555d555e8,0,(nil)) = 0x0000555556088000
26468 rt_sigprocmask(SIG_BLOCK,NULL,0x00007cb49a360d60,8) = 0
--- SIGSEGV {si_signo=SIGSEGV, si_code=2, si_addr=0x0000555556088000} ---
26468 rt_sigprocmask(SIG_SETMASK,0x00007cb49a360d60,NULL,8) = 0
26468 write(2,0x55d55120,67)mmap05.c:63: TPASS: Received SIGSEGV
signal as expected
= 67
26468 munmap(0x0000555556088000,4096) = 0
26468 getpid() = 26468
26468 wait4(-1,0x555555d5557c,0,(nil)) = -1 errno=10 (No child processes)
26468 clock_gettime(CLOCK_MONOTONIC,0x00007cb49a37a558) = 0 ({tv_sec = 7997,tv_nsec = 918056624})
26468 getppid() = 26466
26468 getppid() = 26466
26468 kill(26466,SIGUSR1) = 0
26468 close(3) = 0
= -1 errno=512 (To be restarted)
--- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=26468, si_uid=1000} ---
26466 setitimer(ITIMER_REAL,{it_interval={tv_sec = 0,tv_usec = 0},it_value={tv_sec = 30,tv_usec = 0}},0x0000555555d54e58) = 0 (old_value = {it_interval={tv_sec = 0,tv_usec = 0},it_value={tv_sec = 29,tv_usec = 999051}},)
26466 rt_sigreturn(0x1e) = -1 errno=513 (Successful exit from sigreturn)
26466 wait4(26468,0x555555d555e8,0,(nil))26468 exit_group(0)
= 26468
```


3.3 经验总结——用户态调试经验



由于总是在测例中出现usertrap，或行为更诡异、在内核中无法直接观察的错误，我们需要使用gdb进入用户态调试

自行编译了带符号表的glibc动态链接启动程序、musl库函数、busybox便于gdb定位

```
kidszz@DESKTOP-C4GOCEO:~/F7LY/tmp$ ls
ld-linux-riscv64-lp64d.so.1.unstripped  libc.so.6.unstripped
kidszz@DESKTOP-C4GOCEO:~/F7LY/tmp$ file ld-linux-riscv64-lp64d.so.1.unstripped
ld-linux-riscv64-lp64d.so.1.unstripped: ELF 64-bit LSB shared object, UCB RISC-V,
RVC, double-float ABI, version 1 (SYSV), dynamically linked, BuildID[sha1]=c779d
6eb2d438e3178484f37976cbc8794232379, with debug_info, not stripped
kidszz@DESKTOP-C4GOCEO:~/F7LY/tmp$ file libc.so.6.unstripped
libc.so.6.unstripped: ELF 64-bit LSB shared object, UCB RISC-V, RVC, double-float
ABI, version 1 (GNU/Linux), dynamically linked, interpreter /lib/ld-linux-riscv6
4-lp64d.so.1, BuildID[sha1]=a729c303efc42f819c52b0913cefe55789f0ebcf, for GNU/Lin
ux 4.15.0, with debug_info, not stripped
```

```
kidszz@DESKTOP-C4GOCEO: x + -
[allocate_vma_page] successfully mapped page at va=0x000000000ad000, pa=0x
000000000af000, pte_flags=0x1a
mmap_handler: found VMA 3 for va 0x000000000ad2a8
mmap_handler: store page fault at 0x000000000ad2a8
[allocate_vma_page] readable page at 0x000000000ad2a8
[allocate_vma_page] writable page at 0x000000000ad2a8
[allocate_vma_page] offset: 1196032, file_size: 1209480
[allocate_vma_page] reading from file /glibc/Lib/Libc.so.6 at offset 1196032
(file_size=1209480)
normal file offset: 832, requested offset: 1196032
[allocate_vma_page] successfully mapped page at va=0x000000000ad000, pa=0x
000000000b0000, pte_flags=0x16
mmap_handler: found VMA 0 for va 0x000000000980d4
mmap_handler: load page fault at 0x000000000980d4
[allocate_vma_page] readable page at 0x000000000980d4
[allocate_vma_page] offset: 65536, file_size: 1209480
[allocate_vma_page] reading from file /glibc/Lib/Libc.so.6 at offset 65536 (
file_size=1209480)
normal file offset: 832, requested offset: 65536
[allocate_vma_page] successfully mapped page at va=0x00000000098000, pa=0x
000000000b1000, pte_flags=0x1a
mmap_handler: found VMA 0 for va 0x000000000a3f89
mmap_handler: load page fault at 0x000000000a3f89
[allocate_vma_page] readable page at 0x000000000a3f89
[allocate_vma_page] offset: 110592, file_size: 1209480
[allocate_vma_page] reading from file /glibc/Lib/Libc.so.6 at offset 110592
(file_size=1209480)
normal file offset: 832, requested offset: 110592
[allocate_vma_page] successfully mapped page at va=0x000000000a3000, pa=0x
000000000b2000, pte_flags=0x1a

../sysdeps/generic/dl-protected.h
39 access. */
40 _dl_signal_error (0, map->l_name, undef_name,
41 N_("copy relocation against non-copyab
42 else if (ref->st_value != 0
43 && ref->st_shndx == SHN_UNDEF
44 && (type_class & ELF_RTYPE_CLASS_PLT))
45 /* Disallow non-zero symbol values of undefined symbols
46 executable, which are used as the function pointer, a
47 protected function symbols in a shared object with in
48 external access. */
49 _dl_signal_error (0, map->l_name, undef_name,
50 N_("non-canonical reference to canonic

0x4c8d2 <do_lookup_x+54> sd a7,64(sp)
0x4c8d4 <do_lookup_x+56> sd a4,56(sp)
0x4c8d6 <do_lookup_x+58> mv s6,t5
0x4c8d8 <do_lookup_x+60> ld t3,0(a5)
0x4c8dc <do_lookup_x+64> srli a5,a6,0x3
0x4c8e0 <do_lookup_x+68> srli a4,a1,0x6
0x4c8e4 <do_lookup_x+72> add s10,t3,a5
>0x4c8e8 <do_lookup_x+76> andi a5,s11,1
0x4c8ec <do_lookup_x+80> sd a5,120(sp)
0x4c8ee <do_lookup_x+82> li a5,-1
0x4c8f0 <do_lookup_x+84> srli a5,a5,0x20

remote Thread 1.1 (src) In: do_lookup_x L44 PC: 0x4c8e8
(gdb) ni
(gdb) ni
(gdb) si
do_lookup_x (undef_name=undef_name@entry=0xa3f89 "res",
new_hash=new_hash@entry=2090015214, old_hash=old_hash@entry=0x876d8,
ref=0x980d0, result=result@entry=0x876e8, scope=0x67560, i=0,
version=version@entry=0x1bb0c0, flags=flags@entry=9,
skip=skip@entry=0x0, type_class=type_class@entry=0,
undef_map=undef_map@entry=0x67a60) at ./elf/dl-lookup.c:363
0x00000000004c8e8 in _dl_check_protected_symbol (
type_class=<optimized out>, map=<optimized out>, ref=<optimized out>,
undef_map=<optimized out>, undef_name=<optimized out>)
at ../sysdeps/generic/dl-protected.h:44
(gdb) |
```

核心技术：进入后查看函数调用栈、寄存器值，对比发现问题指令



感谢垂听！

欢迎交流、指正